



COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

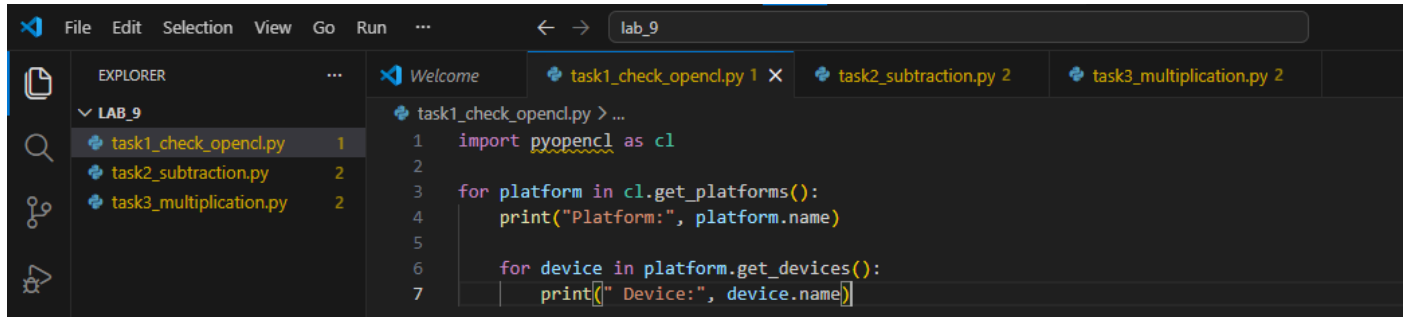
SUBMITTED BY

NAME : RIFAQ AJMAL
REG NO : 23MDBCS 435
SECTION : CS-A
SEMESTER : 6th

Date: 15/4/2026

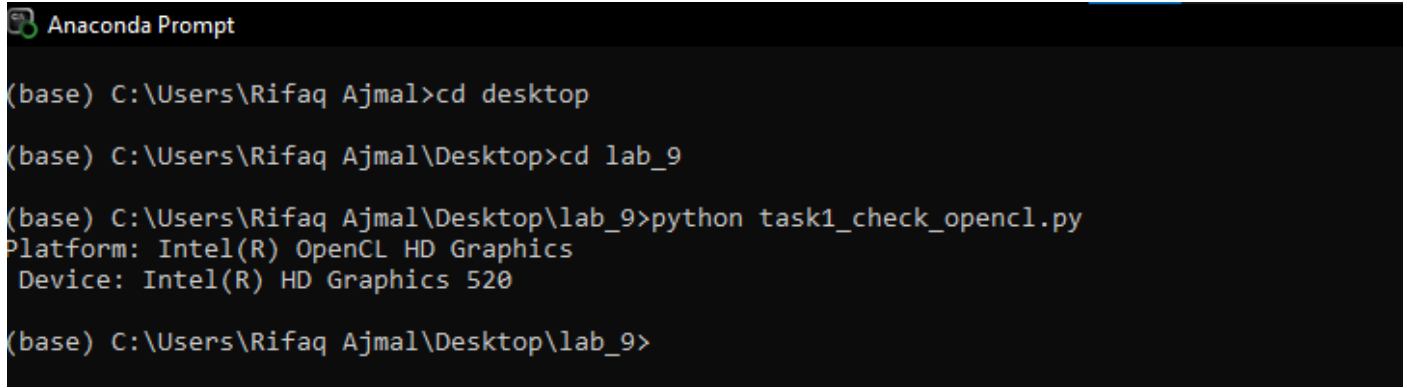
Lab 09

Task 1



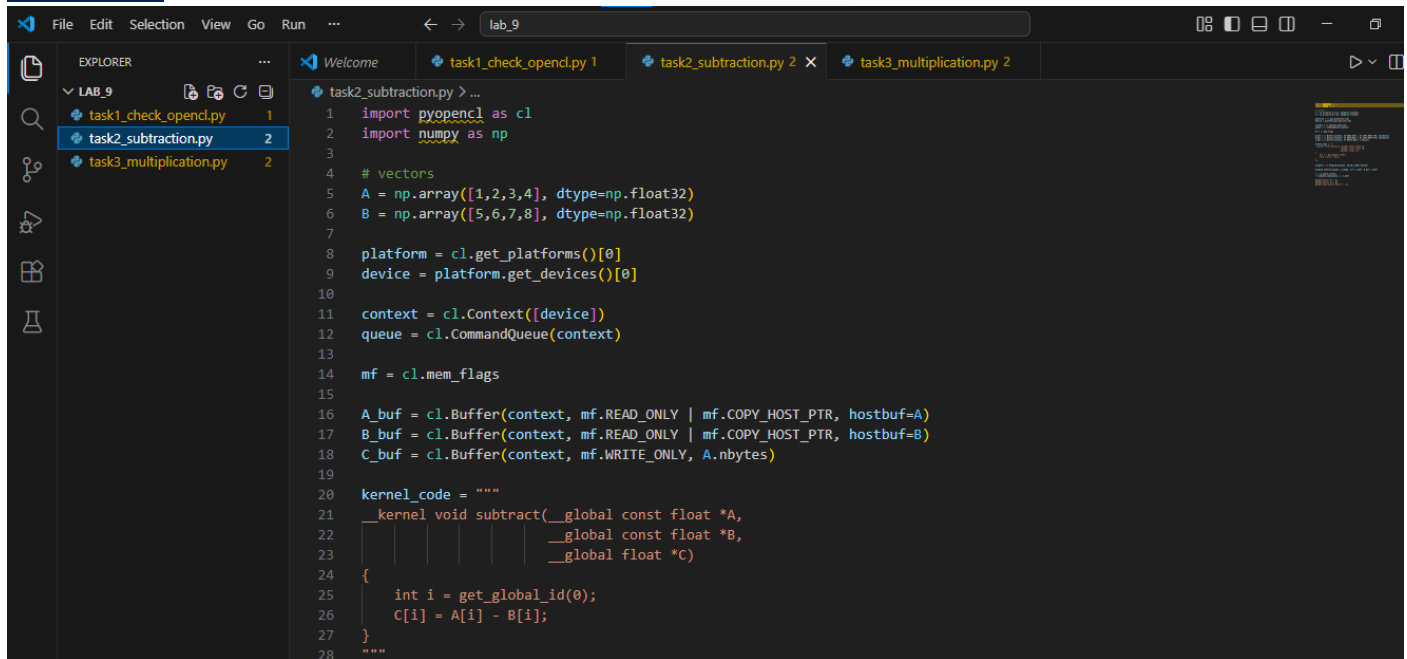
```
1 import pyopengl as cl
2
3 for platform in cl.get_platforms():
4     print("Platform:", platform.name)
5
6     for device in platform.get_devices():
7         print("Device:", device.name)
```

Output



```
(base) C:\Users\Rifaq Ajmal>cd desktop
(base) C:\Users\Rifaq Ajmal\Desktop>cd lab_9
(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>python task1_check_opengl.py
Platform: Intel(R) OpenCL HD Graphics
Device: Intel(R) HD Graphics 520
(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>
```

Task 2



```
1 import pyopengl as cl
2 import numpy as np
3
4 # vectors
5 A = np.array([1,2,3,4], dtype=np.float32)
6 B = np.array([5,6,7,8], dtype=np.float32)
7
8 platform = cl.get_platforms()[0]
9 device = platform.get_devices()[0]
10
11 context = cl.Context([device])
12 queue = cl.CommandQueue(context)
13
14 mf = cl.mem_flags
15
16 A_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=A)
17 B_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=B)
18 C_buf = cl.Buffer(context, mf.WRITE_ONLY, A.nbytes)
19
20 kernel_code = """
21 _kernel void subtract(__global const float *A,
22                     __global const float *B,
23                     __global float *C)
24 {
25     int i = get_global_id(0);
26     C[i] = A[i] - B[i];
27 }
28 """
```

```

29
30 program = cl.Program(context, kernel_code).build()
31
32 program.subtract(queue, A.shape, None, A_buf, B_buf, C_buf)
33
34 C = np.empty_like(A)
35 cl.enqueue_copy(queue, C, C_buf)
36
37 print("Vector A:", A)
38 print("Vector B:", B)
39 print("Subtraction Result:", C)

```

Output

```

(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>python task2_subtraction.py
Vector A: [1. 2. 3. 4.]
Vector B: [5. 6. 7. 8.]
Subtraction Result: [-4. -4. -4. -4.]

(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>_

```

Task 3

```

1 import pyopencl as cl
2 import numpy as np
3
4 A = np.array([1,2,3,4], dtype=np.float32)
5 B = np.array([5,6,7,8], dtype=np.float32)
6
7 platform = cl.get_platforms()[0]
8 device = platform.get_devices()[0]
9
10 context = cl.Context([device])
11 queue = cl.CommandQueue(context)
12
13 mf = cl.mem_flags
14
15 A_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=A)
16 B_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=B)
17 C_buf = cl.Buffer(context, mf.WRITE_ONLY, A.nbytes)
18
19 kernel = """
20 __kernel void multiply(__global const float *A,
21                       __global const float *B,
22                       __global float *C)
23 {
24     int i = get_global_id(0);
25     C[i] = A[i] * B[i];
26 }
27 """
28
29 program = cl.Program(context, kernel).build()
30
31 program.multiply(queue, A.shape, None, A_buf, B_buf, C_buf)
32
33 C = np.empty_like(A)
34 cl.enqueue_copy(queue, C, C_buf)
35
36 print("Result:", C)

```

Output

```

(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>python task3_multiplication.py
Result: [ 5. 12. 21. 32.]

(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>

```

Task 4

The screenshot shows a code editor with a dark theme. The Explorer panel on the left shows a project named 'LAB_9' containing four files: 'task1_check_opengl.py', 'task2_subtraction.py', 'task3_multiplication.py', and 'task4_temp_array.py'. The 'task4_temp_array.py' file is selected and its code is displayed in the main editor. The code imports 'pyopencl as cl' and 'numpy as np'. It creates two NumPy arrays, 'A' and 'B', with shapes (2,4,6) and (3,5,7) respectively, both of dtype 'float32'. It then initializes a PyOpenCL context and queue on a specific device. Memory flags are set for buffers. A kernel named 'temp_mul' is defined with a C-style function signature and a body that calculates the product of corresponding elements in 'A' and 'B'. The kernel is built and executed, with the result stored in a buffer. Finally, the result is copied back to a NumPy array and printed.

```
1 import pyopencl as cl
2 import numpy as np
3
4 A = np.array([2,4,6], dtype=np.float32)
5 B = np.array([3,5,7], dtype=np.float32)
6
7 platform = cl.get_platforms()[0]
8 device = platform.get_devices()[0]
9
10 context = cl.Context([device])
11 queue = cl.CommandQueue(context)
12
13 mf = cl.mem_flags
14
15 A_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=A)
16 B_buf = cl.Buffer(context, mf.READ_ONLY | mf.COPY_HOST_PTR, hostbuf=B)
17 temp_buf = cl.Buffer(context, mf.WRITE_ONLY, A.nbytes)
18
19 kernel = """
20 __kernel void temp_mul(__global const float *A,
21                       __global const float *B,
22                       __global float *temp)
23 {
24     int i = get_global_id(0);
25     temp[i] = A[i] * B[i];
26 }
27 """
28
29 program = cl.Program(context, kernel).build()
30
31 program.temp_mul(queue, A.shape, None, A_buf, B_buf, temp_buf)
32
33 temp = np.empty_like(A)
34 cl.enqueue_copy(queue, temp, temp_buf)
35
36 print("Temporary array:", temp)
```

Output

```
(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>python task4_temp_array.py
Temporary array: [ 6. 20. 42.]

(base) C:\Users\Rifaq Ajmal\Desktop\lab_9>
```